SLIDE 01

Two problems

AI Changes the Delivery System and the Product Itself

Good afternoon. I am Vitalii. My career spans embedded systems integration, testing and project management. Today I lead PMO and delivery. I built their foundations in my company from scratch.

I also stay hands-on with AI. I believe its impact could be comparable to writing and printing. Writing moved memory outside our heads. Ideas could outlive their authors and change institutions.

Now we delegate parts of thinking: interpretation, planning and choice. I do not mean consciousness.

Such a shift requires clear problem statements. Market narratives can obscure them: first developers will disappear; then code review is the bottleneck to remove.

I see two problems.

First, AI can unbalance the SDLC. Adoption means rebalancing the system.

Second, delegating decisions inside a product requires a control architecture. Complex, high-impact systems may need R&D and a socio-technical design before we can promise delivery.

In both cases, what must we remain able to understand and control?

SLIDE 02

The whole delivery system

Let's use the theory of constraints.

Imagine each delivery step handles eight comparable tasks a month. Now coding produces one hundred. Review still handles eight. If we admit everything, ninety-two tasks wait.

To use the extra capacity, we must examine downstream steps, including operations, and upstream intent and design. This can involve the whole business, including marketing and sales. We can also limit incoming work. We do not have to use every unit of generation capacity.

Suppose we rebalance everything. Beyond operations is the user.

Someone opens our app twice a month. They can absorb eight changes. One hundred visible changes may overwhelm them, even if each is useful. Faster delivery could reduce retention and revenue.

We can release changes gradually. But the point remains: the system includes the user's ability to absorb them.

These steps also exchange knowledge, feedback and artifacts. Overload affects those connections too.

If that imbalance remains, overloaded downstream teams can miss errors and pass them on. Both the SDLC and its product can deteriorate: team burnout, less reliable delivery, poorer product quality and stability.

So, a supposedly safe AI rollout for a mid-sized or larger engineering organization: buy licenses, add a little training. What could possibly go wrong? The system-level problem was never recognized, named or assigned an owner.

SLIDE 03

Review and ownership

Review has two functions: a quality gate and a way for the team to understand the system.

Remove either without a replacement, and local speed can create future costs.

Understanding supports ownership: the ability and authority to change, diagnose and recover the system. Responsibility remains even when that ability weakens.

One experiment involved 69 participants, including fourteen professional developers. With AI, they completed more work but answered fewer questions correctly about their code. It measured short individual tasks, not whole-team ownership.

AI can also help people learn. We must check what our process actually does.

Imagine buying licenses and courses for a thousand engineers. That does not establish whether delivery can absorb more work.

Queues can grow. Review pressure can rise. Weaker gates can let more defects and debt reach production.

That creates a risk of technical default: responsibility for a system we can no longer maintain or recover at an acceptable cost. It is a risk to investigate, not a proven outcome for every organization.

SLIDE 04

The evidence

These reports examine different populations and outcomes. They do not form one pipeline, and not all are independent of vendors.

NBER studied over half a million developers. Estimated commits reached about three point four times baseline; releases, one point three. Samples and tool coverage differ. Marketplace supply grew without a matching increase in overall usage. More code alone does not establish more user value.

Chen and Stratton studied 718 firms. Pull requests per active worker-month rose about 23 percent after agent adoption. Submission-to-merge time rose about 49 percent. That includes waiting.

Agarwal and colleagues studied 1,197 repositories, including controls. Code complexity increased. Warnings increased significantly in the agent-first group, but not the IDE-first group.

DORA links AI adoption with higher throughput and greater instability. It is a survey, not proof of causality.

GitClear analyzed 623 million changes. It reports fewer refactoring-related line moves and more duplicated blocks. These are quality signals, not proof that AI caused defects.

My conclusion: assess the whole delivery system, beyond coding activity.

SLIDE 05

Finding a new balance

AI adoption changes the system that earns your revenue. Licenses and courses are only inputs.

Across experiments, watch code quality, queues and recovery. Limit incoming work. Compare useful outcomes and full cost with a baseline.

Review agents may help. But generator and reviewer can share blind spots. And who learns how the system changed?

Generated documentation may help too. But a clear explanation can describe a poor decision. Who checks it against code, tests and actual behavior?

Even accurate documentation does not guarantee that the team can use it during an incident.

Disposable code also has limits: data, integrations and obligations can outlive an implementation.

Experiment, but preserve a recovery path when agents fail or become unavailable. Responsibility remains with us.

We may find different operating models rather than one universal AI-native SDLC. Domain, legacy, team, risk and cost will shape AI's role at each step.

SLIDE 06

Decisions inside the product

Now the product itself. An LLM can choose a path to the user's goal at runtime.

For a deterministic component, the same inputs, state and version give the same result. With a model, we often consider a range of possible outputs. Each run reveals one outcome.

Traditional software also has uncertainty. Here, we delegate more decisions inside the application.

Requirements must describe acceptable outcomes and boundaries. For an account-recovery assistant: useful advice, recoverable mistakes, prohibited disclosures and actions requiring authorization.

A bug can be a plausible answer outside those boundaries. Testing needs repeated samples and interpretation of failures. One success proves little about reliability. One serious failure can block release.

Readiness may mean being ready for a feasibility study, not a full feature. Completing an increment and approving its operating scope are separate decisions.

For DevOps and SRE, availability is not enough. We must observe whether the system produces acceptable outcomes. Provider updates and changing context can alter behavior without a code change.

I call these thinking systems because we delegate cognitive work to them. They create new opportunities and new operating demands.

SLIDE 07

The Product Includes Its Control System

Design the control architecture while framing the project. A diagram cannot guarantee reliability. We need evidence that the controls work.

The loop must observe outcomes, compare them with goals and limits, decide and intervene, then verify that the correction worked.

Enforce sensitive data access and account permissions outside the model. Check text output too: a disclosure needs no tool call.

Where automation is insufficient, trained humans may need to observe or decide. Their skills, workload, information and authority become architectural dependencies. This is a socio-technical system.

Include models, monitoring, human review and recovery in the cost. Compare a simpler viable option. If the whole system is not viable, reduce scope or fund more R&D. Complex cases may need a control engineer or mathematician. Sometimes we should not proceed.

For teams building their first agentic applications, a successful demo alone does not justify funding full development. The control architecture can reveal missing expertise, change the risk landscape and improve the budget forecast before we commit.

SLIDE 08

Validation Continues During Operation

Imagine two hundred test responses: 196 acceptable, three wrong instructions and one privacy leak.

The score is 98 percent. But that disclosure is unacceptable in this project. We must redesign the affected path. The average cannot make that decision for us.

Suppose we fix it and release within an agreed scope. Users, context or the model can still change. Yesterday's evidence may no longer describe today's behavior.

Our claim that the system works remains a hypothesis in production.

Shadow operation and canaries can provide evidence where exposure is acceptable. They cannot replace preventive controls. Rollback cannot undo a disclosure.

For DevOps and SRE, monitor outcomes, controls and response capacity. Beyond availability, track qualified coverage, queues, workload and response time. Check whether people have the information and authority to act, and whether interventions work. Fatigue is harder to measure directly; workload can signal strain.

An operator can be online while their control is already failing.

If capacity falls, reduce autonomy, use a tested, staffed fallback or pause.

SLIDE 09

Closing

You may recognize this: AI spending rises, more code appears, but the business result is unclear. Or the demo works, but nobody knows who will intervene in production, how quickly, and at what cost.

These are the problems I research. Both repositories use systems thinking: examining the whole system and its relationships.

The Subprime Code Crisis looks at SDLC imbalance through the theory of constraints. It is an early research repository, still a little rough. If these or other problems with AI adoption affect your teams, I'd be glad to discuss them with you.

Uncertainty Architecture looks at thinking systems through control theory. This repository explores the control problem we discussed. If you have faced it, I'd be glad to hear about your experience too.

Both projects are public and developing. The QR code leads to my website, articles and contact details.

Come and talk to me afterwards.

Let's build the future. But keep asking: what if not? How will we know, and can we still act?

Thank you.

Delivery notes - not spoken

Обсяг сценічного тексту: **1 530 слів** проти **1 552** у R13; на **22 слів менше**. Пораховано лише текст дев'яти сценічних секцій, без заголовків, службових нотаток і джерел. Однаковий для обох версій підрахунок: слова з літерами/цифрами, внутрішнім дефісом або апострофом (\b\w+(?:[-’’]\w+)*\b). Числа й абревіатури при вимові можуть трохи збільшити тривалість.

Темп чистого мовлення	Без додаткових пауз	Із запасом 2-3 хв на паузи, схеми та перемикання
80 слів/хв	19:08	21:08-22:08
90 слів/хв	17:00	19:00-20:00
100 слів/хв	15:18	17:18-18:18
110 слів/хв	13:55	15:55-16:55
120 слів/хв	12:45	14:45-15:45

Це сценарії розрахунку, не результат репетиції. Якщо вимірюєш загальний темп запису разом із паузами, не додавай ті самі паузи вдруге. Бюджет 20-22 хв - верхня межа, яку не потрібно заповнювати словами заради тривалості. Загальний ліміт планування: до 23 хв виступу та 7 хв Q&A в 30-хвилинному слоті.

Читай повністю вголос, з реальними паузами та перемиканням слайдів. Не оцінюй час читанням подумки. Якщо виходить 16-18 хвилин, це нормальний результат: не додавай назад повтори лише для заповнення слоту.

- **Слайд 2:** «What could possibly go wrong?» - легка іронія та коротка пауза.

- **Слайд 4:** називай лише цифри зі спічу. Інші числа залишаються референсами для слухачів.

- **Слайд 6:** покажи формули, але не читай їх як математичну лекцію.

- **Слайд 7:** веди слухача від контрольного циклу до людських залежностей і рішення про проєкт. Пауза після «architectural dependencies». Не прискорюй фінал про ризики, експертизу та бюджет до фінансування повної розробки. Схема містить ширшу карту питань; усі підписи читати не потрібно.

- **Слайд 8:** від прикладу 98% перейди до того, що потрібно спостерігати й коли втручатися. Пауза після «An operator can be online while their control is already failing». Заверши дією: зменшити автономність, перейти на перевірений і забезпечений людьми fallback або призупинити роботу.

- **Слайд 9:** пауза після «the business result is unclear».

Дуже короткі речення не слід вимовляти однаковим уривчастим ритмом: з'єднуй речення одного абзацу природною інтонацією. Мета - проста мова і час на розуміння.

Sources - not spoken

All six research references and their qualifications are retained from R13. Research evidence is now on Slide 04; the ownership experiment is on Slide 03. This is an editorial alignment to the existing R9 deck, not a new research review.

- NBER: Writing Code vs. Shipping Code. Different samples and tool coverage; activity, releases and marketplace use are different outcomes. The 25.5× changed-LOC figure on the existing slide is not spoken and must not be described as developer speed.

- Chen and Stratton: Artificial Intelligence in the Firm. 718 firms; PRs per active worker-month; elapsed submission-to-merge time includes waiting.

- Agarwal, He and Vasilescu. 1,197 repositories includes treated and matched controls; warnings are significant only for the agent-first group in Table 2. Complexity and warnings are proxies.

- DORA 2025. Google Cloud program; survey associations, not causal proof. The existing slide gives 4,867 respondents.

- GitClear: The Maintainability Gap. Vendor analysis of 623 million code changes; structural signals do not isolate AI causality.

- Martin-Lopez and colleagues: More Code, Less Understanding?. 69 participants, 14 professionals; short individual tasks, not whole-team production ownership.

Що змінено в R14 - не для сцени

- **Порядок:** 1, 2, 4, 3, 5, 6, 8, 7, 9 відносно R13. Review/ownership тепер на слайді 3, evidence - на 4; architecture/planning - на 7, runtime - на 8.

- **Сім блоків без переписування:** поточні слайди 1-6 та 9 зберігають повний сценічний текст R13 дослівно; у 3-4 змінилися лише позиції.

- **Слайд 7:** архітектура під час постановки проєкту; спостереження, рішення, своєчасна дія та перевірка результату; люди як умовна архітектурна залежність; повна вартість, простіша життєздатна альтернатива й рішення до зобов'язання щодо бюджету.

- **Слайд 8:** збережено приклад 196/3/1 та межі shadow/canary/rollback. Сюди перенесено моніторинг людської спроможності й дії при її падінні. Явно названо outcomes, controls and response capacity; fallback має бути перевірений і забезпечений людьми.

- **Службові нотатки:** оновлено номери слайдів, акценти, обсяг і розрахунковий час. Наявність тексту не є доказом репетиції.

R14 - окрема редакція сценічного спічу для R9, не нова редакція слайдів. Історичний R13 і весь пакет R9 залишаються незмінними. Для цієї версії виступу використовуй R14; вкладені нотатки історичного R9 містять попередній спіч і розширені референси.